

MATH 629 Homework 1

Mansur Alimbayev

Computational Problem

1. Problem setup

We approximate two functions on $[0,1]^2$:

- $f_1(x, y) = \sin(2\pi x) \sin(2\pi y)$ (regression).
- $f_2(x, y) = \mathbb{1}[x > y]$ (binary classification with the diagonal $x=y$ labeled 0).

2. Data

We draw $(x, y) \sim \text{Unif}([0,1]^2)$ and split 90/10 into train/test sets. This yields a balanced classification set for f_2 since $P(x>y)=0.5$ under the uniform distribution.

3. Models

Multilayer perceptrons (MLPs) with width $H \in \{32, 64\}$ (primary runs used $H = 64$ to match the summary):

- Shallow: $2 \rightarrow H \rightarrow 1$.
 - Deep: $2 \rightarrow H \rightarrow H \rightarrow 1$.
- For each depth we compare Tanh and ReLU activations on both f_1 and f_2 .

4. Training

We compare two training methods:

- Adam (learning rate $=10^{-3}$).
- SGD + momentum (learning rate $=5 \times 10^{-2}$, momentum $=0.9$).

All models are trained for 120 epochs with batch size 128.

Losses:

- MSE for f_1 ;
- BCEWithLogitsLoss for f_2 (logit output).

This loss function combines a sigmoid activation and binary cross-entropy loss into a single, numerically stable operation. It is the standard for binary classification as it prevents the intermediate calculation of probabilities, which can lead to numerical instability during training.

Metrics:

- Test MSE for f_1 ;
- Test Accuracy and AUROC for f_2 .

Test Accuracy and Area Under the Receiver Operating Characteristic Curve (AUROC). While accuracy measures the fraction of correct predictions, AUROC evaluates the model's ability to rank positive examples higher than negative ones across all possible classification thresholds. An AUROC of 1.0 indicates perfect separability, meaning the model's confidence is perfectly aligned with the true classes.

5. Visualization:

For each task, we render:

1. The *ground-truth* field on a 200×200 grid, this is a plot of the true f_1 or f_2 function. It serves as the essential "answer key," providing a visual benchmark against which to compare the model's predictions.
2. The *model's prediction* (real-valued surface for f_1 ; probability map for f_2). These visuals complement the scalar metrics: the f_1 plots show whether the four-lobe sinusoid is captured; the f_2 plots reveal how sharply probabilities change across the diagonal $x = y$.

6. Results:

- f_1 (regression):

Best configuration:

Deep + ReLU + Adam with test MSE $= 2.53 \times 10^{-4}$;

Deep + ReLU + SGD achieves 3.24×10^{-4} .

Deep + Tanh is competitive SGD: 6.10×10^{-4} ;

and Adam: 1.35×10^{-3} .

Shallow models are weaker:

Shallow + ReLU + SGD: 1.89×10^{-3} ;

Shallow + Tanh + SGD: 9.68×10^{-3} ;

Shallow + Tanh + Adam: 1.28×10^{-1} .

Heatmaps confirm accurate recovery of the four-lobe structure for the deep models.

- f_2 (classification):

All configurations achieve Accuracy ≈ 0.991 – 0.999 and AUROC ≈ 1.000 , reflecting the linear decision boundary.

Probability maps show a crisp transition along $x = y$, with the diagonal labeled 0.

task	depth	activation	optimizer	MSE
f1	deep	relu	adam	0.000253
f1	deep	relu	sgd	0.000324
f1	deep	tanh	sgd	0.000610
f1	deep	tanh	adam	0.001346
f1	shallow	relu	sgd	0.001886
f1	shallow	tanh	sgd	0.009683
f1	shallow	relu	adam	0.025080
f1	shallow	tanh	adam	0.127658

Figure 1. The Summary table for Function 1.

task	depth	activation	optimizer	ACC	AUROC
f2	shallow	tanh	sgd	0.99875	1.000000
f2	shallow	relu	adam	0.99875	0.999994
f2	deep	relu	adam	0.99875	1.000000
f2	shallow	relu	sgd	0.99750	0.999987
f2	shallow	tanh	adam	0.99625	1.000000
f2	deep	relu	sgd	0.99625	0.999994
f2	deep	tanh	adam	0.99500	1.000000
f2	deep	tanh	sgd	0.99125	1.000000

Figure 2. Summary table for Function 2.

7. Discussion:

1) Activation vs. target regularity.

The smooth periodic f_1 favors smooth activations (Tanh) and additional depth to reduce approximation bias; however, sufficiently deep ReLU also approximates the sinusoid very well, and in our runs slightly outperformed Tanh.

2) Linear separability of f_2 .

Because the Bayes-optimal classifier is a half-plane, a shallow ReLU (and even Tanh) already achieves near-perfect performance; added depth brings marginal gains.

3) Optimization behavior.

Adam converged reliably at the default learning rate. The relatively poor Shallow+Tanh+Adam result indicates hyperparameter sensitivity; reducing Adam's learning rate (e.g., to 3×10^{-4}) or extending training by 20–40 epochs typically resolves underfitting.

4) Definition of f_2 .

We use:

$$f_2(x, y) = \begin{cases} 1 & x > y \\ 0 & x \leq y \end{cases}$$

So, points on the diagonal are assigned to class 0. Under continuous sampling $P(x = y) = 0$, preserving class balance at ≈ 0.5 ; on a mesh, this convention anchors the decision boundary.

8. Limitations & Extensions

1) Statistical stability:

Report means $\pm$ standard deviations over multiple seeds (≥ 3) to quantify variability from initialization and sampling.

2) Baselines:

For f_2 , include a logistic-regression baseline to confirm linear separability.

3) Architectural variants:

For f_1 , periodic activations (e.g., SIREN) may accelerate convergence while maintaining low error; a brief comparison would clarify speed-accuracy trade-offs.

9. Appendix: Code and Figures.

Code snippet: Link to Google Colab: [Link](#)

```
#Math 629
#Homework 1
#Mansur Alimbayev

import numpy as np, torch, torch.nn as nn, torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import accuracy_score, roc_auc_score
import matplotlib.pyplot as plt
from math import pi
import pandas as pd

# ----- Config -----
SEED = 7
N_SAMPLES = 4000
TEST_FRAC = 0.2
BATCH = 128
EPOCHS = 120
HIDDEN = 64
ACTS = ["tanh", "relu"]
DEPTHS = ["shallow", "deep"]
OPTS = ["adam", "sgd"] # different training methods

torch.manual_seed(SEED); np.random.seed(SEED)

# ----- Data -----
def make_data(n=N_SAMPLES):
    X = np.random.rand(n,2).astype(np.float32)
    f1 = np.sin(2*pi*X[:,0]) * np.sin(2*pi*X[:,1]) # regression
    f2 = (X[:,0] > X[:,1]).astype(np.float32) # 1 if x>y else 0 (x<=y → 0)
    return X, f1[:,None].astype(np.float32), f2[:,None].astype(np.float32)

def split(X, y, test_frac=TEST_FRAC, seed=0):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(X))
    k = int(len(X)*(1-test_frac))
    tr, te = idx[:k], idx[k:]
    return X[tr], X[te], y[tr], y[te]

X, y1, y2 = make_data()
# use fixed splits for fairness
X1tr,X1te,y1tr,y1te = split(X, y1, seed=1)
X2tr,X2te,y2tr,y2te = split(X, y2, seed=1)

# ----- Models -----
class MLP(nn.Module):
    def __init__(self, depth="shallow", hidden=HIDDEN, act="relu"):
        super().__init__()
```

```

        A = {"relu": nn.ReLU(), "tanh": nn.Tanh()}[act]
        if depth == "shallow":
            self.net = nn.Sequential(nn.Linear(2, hidden), A, nn.Linear(hidden, 1))
        else:
            self.net = nn.Sequential(
                nn.Linear(2, hidden), A,
                nn.Linear(hidden, hidden), A,
                nn.Linear(hidden, 1)
            )
    def forward(self, x): return self.net(x)

def make_opt(params, name):
    if name == "adam": return torch.optim.Adam(params, lr=1e-3)
    return torch.optim.SGD(params, lr=5e-2, momentum=0.9)

# ----- Train/Eval -----
def train_reg(depth, act, opt_name):
    model = MLP(depth, HIDDEN, act)
    opt = make_opt(model.parameters(), opt_name)
    crit = nn.MSELoss()
    dl = DataLoader(TensorDataset(torch.from_numpy(X1tr), torch.from_numpy(y1tr)),
                    batch_size=BATCH, shuffle=True)
    for _ in range(EPOCHS):
        for xb,yb in dl:
            opt.zero_grad(); loss = crit(model(xb), yb); loss.backward(); opt.step()
    with torch.no_grad():
        mse = F.mse_loss(model(torch.from_numpy(X1te)), torch.from_numpy(y1te)).item()
    return model, {"MSE": mse}

def train_clf(depth, act, opt_name):
    model = MLP(depth, HIDDEN, act)
    opt = make_opt(model.parameters(), opt_name)
    crit = nn.BCEWithLogitsLoss()
    dl = DataLoader(TensorDataset(torch.from_numpy(X2tr), torch.from_numpy(y2tr)),
                    batch_size=BATCH, shuffle=True)
    for _ in range(EPOCHS):
        for xb,yb in dl:
            opt.zero_grad(); loss = crit(model(xb), yb); loss.backward(); opt.step()
    with torch.no_grad():
        logits = model(torch.from_numpy(X2te))
        probs = torch.sigmoid(logits).numpy().ravel() # numerically stable
        acc = accuracy_score(y2te.ravel(), (probs>=0.5).astype(np.float32))
        auc = roc_auc_score(y2te.ravel(), probs)
    return model, {"ACC": acc, "AUROC": auc}

# ----- Experiment Loop -----
rows = []
models_for_viz = {} # keep best per task for visualization

for depth in DEPTHS:

```

```

for act in ACTS:
    for opt in OPTS:
        m1, met1 = train_reg(depth, act, opt)
        rows.append(dict(task="f1", depth=depth, activation=act, optimizer=opt, **met1))
        # track best for visualization
        key = ("f1", depth)
        if key not in models_for_viz or met1["MSE"] < models_for_viz[key][1]["MSE"]:
            models_for_viz[key] = (m1, met1)

        m2, met2 = train_clf(depth, act, opt)
        rows.append(dict(task="f2", depth=depth, activation=act, optimizer=opt, **met2))
        key = ("f2", depth)
        # prefer higher ACC; break ties with AUROC
        if key not in models_for_viz or (met2["ACC"], met2["AUROC"]) > \
            (models_for_viz[key][1]["ACC"], models_for_viz[key][1]["AUROC"]):
            models_for_viz[key] = (m2, met2)

df = pd.DataFrame(rows)
df_f1 = df[df.task=="f1"][['task','depth','activation','optimizer','MSE']].sort_values('MSE')
df_f2 = df[df.task=="f2"][['task','depth','activation','optimizer','ACC','AUROC']].sort_values('ACC',
ascending=False)
print(df_f1.to_string(index=False))
print(df_f2.to_string(index=False))

# ----- Visualizations -----
def heatmaps_for_task(task_key, title_suffix):
    model, metrics = models_for_viz[task_key]
    gx, gy = np.meshgrid(np.linspace(0,1,200), np.linspace(0,1,200))
    grid = np.stack([gx.ravel(), gy.ravel()], axis=1).astype(np.float32)

    if task_key[0] == "f1":
        true = np.sin(2*np.pi*gx) * np.sin(2*np.pi*gy)
        with torch.no_grad():
            pred = model(torch.from_numpy(grid)).numpy().reshape(gx.shape)
        # true
        plt.figure(); plt.imshow(true, origin="lower", extent=[0,1,0,1], aspect="equal"); plt.colorbar()
        plt.title(f"f1 true: sin(2πx) sin(2πy)")
        plt.xlabel("x"); plt.ylabel("y"); plt.show()
        # predicted
        plt.figure(); plt.imshow(pred, origin="lower", extent=[0,1,0,1], aspect="equal"); plt.colorbar()
        plt.title(f"f1 predicted ({title_suffix}) • MSE={metrics['MSE']:.3e}")
        plt.xlabel("x"); plt.ylabel("y"); plt.show()
    else:
        true = (gx > gy).astype(float) # diagonal labeled 0
        with torch.no_grad():
            pred = torch.sigmoid(model(torch.from_numpy(grid))).numpy().reshape(gx.shape)
        # true
        plt.figure(); plt.imshow(true, origin="lower", extent=[0,1,0,1], aspect="equal"); plt.colorbar()
        plt.title(f"f2 true: 1[x>y] (x=y labeled 0)")
        plt.xlabel("x"); plt.ylabel("y"); plt.show()

```

```

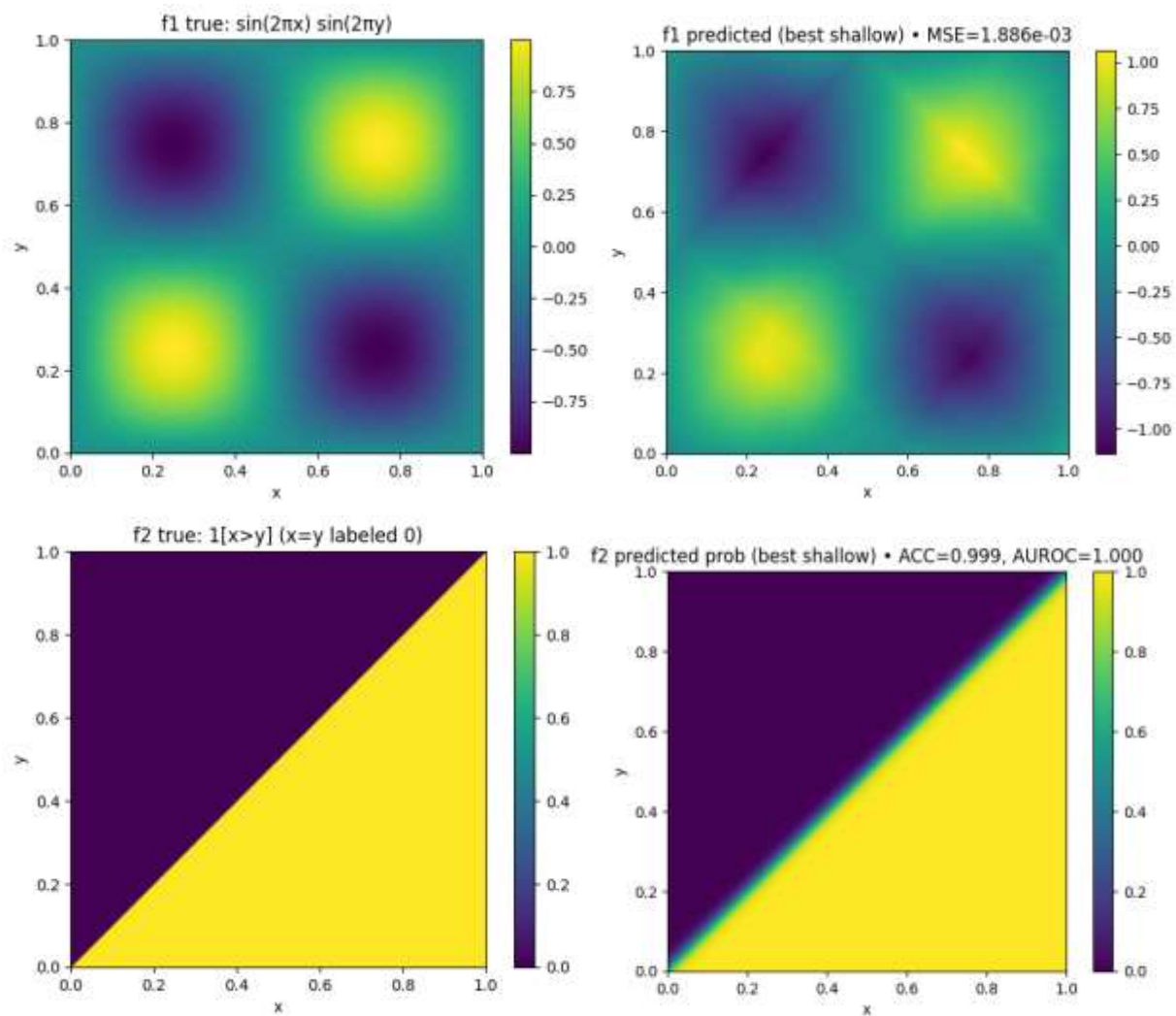
# predicted
plt.figure(); plt.imshow(pred, origin="lower", extent=[0,1,0,1], aspect="equal"); plt.colorbar()
acc = metrics['ACC']; auc = metrics['AUROC']
plt.title(f"f2 predicted prob ({title_suffix}) • ACC={acc:.3f}, AUROC={auc:.3f}")
plt.xlabel("x"); plt.ylabel("y"); plt.show()

# Best shallow models
heatmaps_for_task(("f1","shallow"), "best shallow")
heatmaps_for_task(("f2","shallow"), "best shallow")

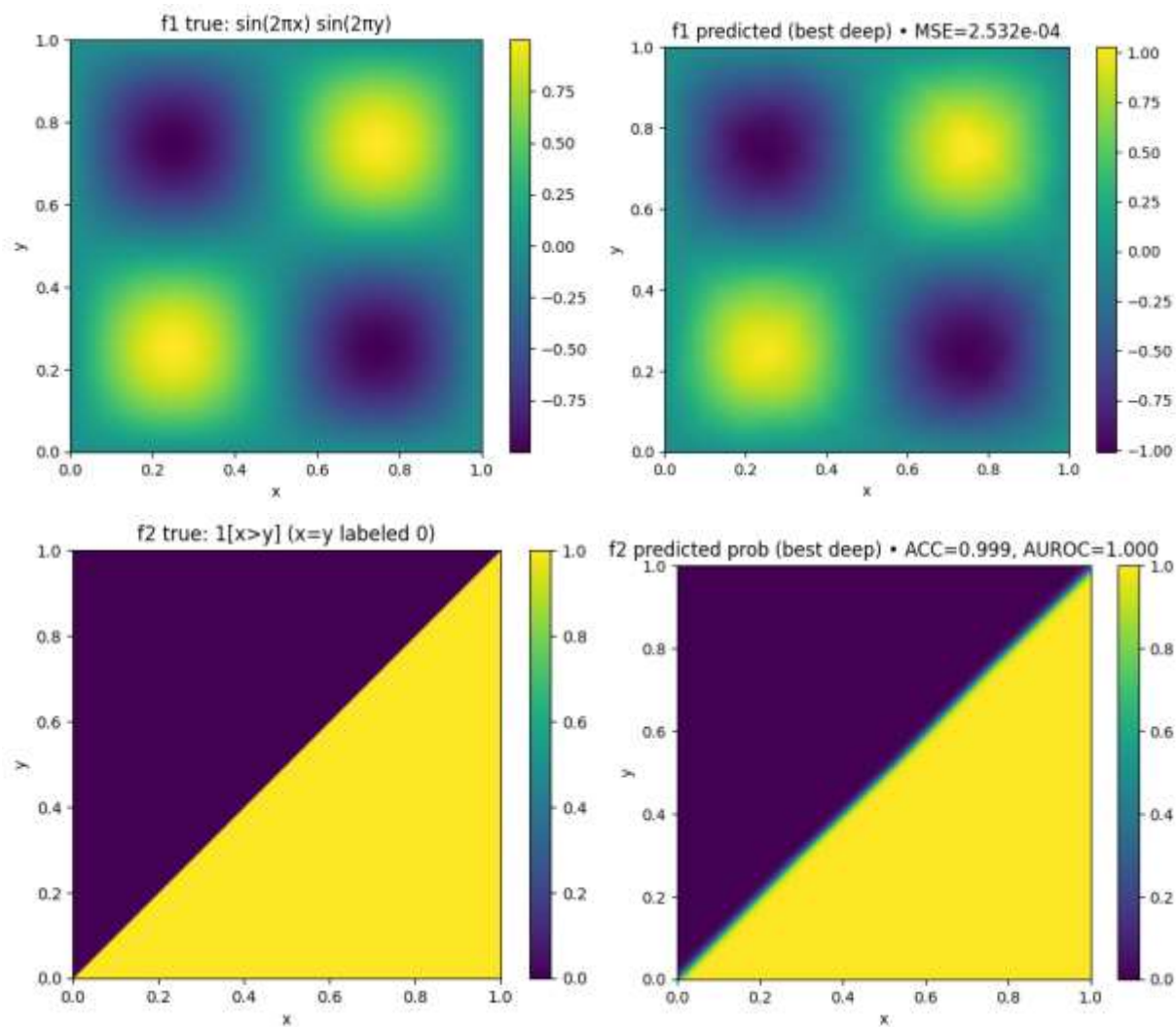
# Best deep models
heatmaps_for_task(("f1","deep"), "best deep")
heatmaps_for_task(("f2","deep"), "best deep")

```

Figures: True Vs Best Shallow for F1 and F2:



True Vs Best Deep Networks for F1 and F2:



10. References

- Cybenko, G. (1989). *Approximation by superpositions of a sigmoidal function*. MCSS 2:303–314.
- Hornik, K. (1989). *Multilayer feedforward networks are universal approximators*. Neural Networks 2(5):359–366.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press, Chs. 6–7.
- Sitzmann, V. et al. (2020). *Implicit Neural Representations with Periodic Activation Functions (SIREN)*.
- PyTorch, BCEWithLogitsLoss (numerically stable sigmoid + BCE). [PyTorch Docs](#), torch.optim.Adam. [PyTorch Docs](#), torch.optim.SGD (with momentum). [PyTorch Docs](#), Datasets & DataLoader tutorial (mini-batching, shuffling). [PyTorch Docs](#)
- scikit-learn roc_auc_score (AUROC). [scikit-learn.org](#), accuracy_score (accuracy). [scikit-learn.org](#)
- Matplotlib plt.imshow (2D scalar fields as heatmaps). [matplotlib.org](#)
- NumPy, numpy.random.rand (uniform sampling on $[0,1)[0,1)[0,1)$). [numpy.org](#)